COMPUTER NETWORKS · CHAPTER-BASED FIELD GUIDE

计算机网络学习指南

从一次网页访问到整个互联网

知识点 · 典型案例 · 抓包实验 · 高频题型 · 14 周路线

零基础入门 | 课程考试 | 408 复习 | 网络编程实践

章节案例版

以协议要解决的真实问题为线索，把五层模型连成一条完整旅程

阅读地图

主线采用自顶向下的学习顺序；需要考试复习时，可把第十一章当作题型索引，把附录 B 当作口头自测答案。

章节	你会解决的问题
第一章 网络全景	分层、交换、封装，以及发送/传播/排队时延
第二章 应用层	HTTP、DNS、缓存、CDN、邮件与 FTP
第三章 传输层	端口、UDP、TCP 字节流、可靠性和拥塞控制
第四章 网络层	IPv4、CIDR、分片、NAT、DHCP、ICMP、IPv6
第五章 路由	DV、LS、Dijkstra、RIP、OSPF 与 BGP
第六章 链路层	以太网、交换机、ARP、CRC 与 VLAN
第七章 物理与无线	信号、容量、复用、Wi-Fi 与 CSMA/CA
第八章 网络安全	密码机制、证书链、TLS 与常见攻击
第九章 综合案例	网页、会议、游戏、数据库和分层排障
第十章 Wireshark	四组实验、过滤器与证据式实验报告
第十一章 高频题型	子网、时延、分片、CRC、窗口、路由
第十二章 学习路线	14 周安排、费曼笔记、项目与毕业标准

导读：怎样把计算机网络真正学会

1. 每个协议都问四个问题

1. **它解决什么问题？** 没有它会发生什么事故？
2. **它的报文长什么样？** 哪些字段决定地址、顺序、状态和错误处理？
3. **双方怎样对话？** 谁先发、谁后发、超时或丢包后怎么办？
4. **它在哪一层工作？** 它依赖谁，又给谁提供服务？

例如，TCP 不是一张“首部字段表”，而是一套解决乱序、丢失、重复、接收方来不及处理和网络拥塞的完整机制。只背字段，很快会忘；从事故倒推机制，知识会自然连起来。

2. 三轮学习法

- **第一轮：看懂故事。** 能用快递、电话、酒店前台等类比，把协议讲给没有基础的人听。
- **第二轮：画出报文。** 能画时序图、封装图、地址变化图，并解释关键字段。
- **第三轮：验证与做题。** 用浏览器开发者工具、命令行和 Wireshark 看到真实报文，再做计算题和综合题。

3. 全书只有一条主线

假设你在浏览器输入 `https://www.example.com/index.html`：

```
应用层：DNS 把域名变成 IP；HTTP 规定请求和响应的格式；TLS 保护通信
   ↓
传输层：TCP 或 QUIC 把数据送到服务器进程，并处理可靠性与拥塞
   ↓
网络层：IP 让数据报跨越多个网络；路由器逐跳选择出口
   ↓
链路层：每一跳都重新装成以太网帧或 Wi-Fi 帧，用 MAC 地址交给下一站
   ↓
物理层：比特最终变成电信号、光信号或无线电波
```

学习过程中始终回到这个场景：这一章解释了旅程中的哪一段？

4. 五层模型速查

层次	核心问题	典型协议/技术	数据单元	主要地址或标识	常见设备
应用层	应用怎样约定业务对话	HTTP、DNS、SMTP、TLS	报文	域名、URL	代理、应用网关
传输层	怎样送到正确进程并控制传输	TCP、UDP、QUIC	报文段/用户数据报	端口、四元组	主机操作系统、负载均衡器
网络层	怎样跨网络到达目的主机	IPv4/IPv6、ICMP、OSPF、BGP	数据报/分组	IP 地址、前缀	路由器
数据链路层	怎样完成相邻节点的一跳交付	Ethernet、Wi-Fi、ARP、VLAN	帧	MAC 地址、VLAN ID	交换机、网卡、AP
物理层	怎样把比特放到介质上	编码、调制、复用	比特	无	中继器、集线器、线缆

关键提醒： 分层是理解工具，不是绝对边界。ARP、TLS、QUIC 等技术经常跨越传统分类；考试时按指定教材口径作答，工程理解则要关注它实际读写了哪些字段、运行在哪里。

第一章 网络全景：分层、交换、封装与时延

1.1 网络由什么组成

互联网可以分成两部分：

- **网络边缘：** 手机、电脑、服务器、摄像头等端系统。应用真正运行在这里。
- **接入网络：** Wi-Fi、以太网、光纤、蜂窝网络等，把端系统接入互联网。
- **网络核心：** 大量路由器和高速链路，负责存储、转发分组。

“自治”很重要：互联网不是一台超级计算机，而是许多独立管理的网络依靠共同协议互联。你家的网络、校园网、运营商网络和云服务商网络属于不同管理域，却都能转发 IP 数据报。

1.2 三种交换方式

方式	做法	优点	缺点	典型场景
电路交换	通信前先建立并独占一条资源通路	时延稳定、资源有保证	空闲时也占资源，利用率低	传统电话网
报文交换	整份报文到达节点后再整体转发	不需专用线路	报文大，缓存和等待时间长	历史电报系统
分组交换	报文切成较小分组，逐个存储转发	共享链路、利用率高、可流水线	会排队、丢包、乱序	互联网

分组交换为什么高效？因为大量用户通常不会同时持续发送。链路可以按需共享。但共享也引出代价：当到达速度超过输出能力，路由器队列增长，排队时延变大，缓存用尽后发生丢包。

1.3 协议与分层

协议有三要素：

- **语法：** 报文格式，例如字段顺序和长度。
- **语义：** 字段代表什么，收到后要采取什么动作。

- **同步/时序：** 报文先后顺序以及超时规则。

分层的价值是把复杂问题拆开。应用层不需要知道光纤怎样调制，只需要调用传输层接口；传输层不需要自己寻找每一台路由器，只需要把报文段交给 IP。代价是会增加首部开销，有时还会出现功能重复或跨层优化。

1.4 封装与解封装

发送端像套信封：

```
HTTP 报文
└ TCP 首部 + HTTP 报文                    = TCP 报文段
   └ IP 首部 + TCP 报文段                 = IP 数据报
      └ 以太网首部 + IP 数据报 + FCS      = 以太网帧
         └ 010101...                       = 物理比特流
```

接收端反向拆封。路由器通常只处理到网络层：收到一帧，去掉旧链路层首尾，检查 IP 首部并选择下一跳，再用下一条链路的格式重新封装。因此：

- 端到端通信中，应用数据会被多层首部包裹。
- 每经过一台路由器，链路层帧都会变化。
- 在没有 NAT、隧道等机制时，源/目的 IP 通常端到端不变；存在 NAT 时 IP 或端口会被改写。

1.5 四类时延

一个分组经过节点时，总时延主要由四部分构成：

1. **处理时延：** 检查首部、校验错误、查转发表。
2. **排队时延：** 在输出队列等待，波动最大。
3. **发送时延：** 把 L 比特推上速率为 R 的链路，`T = L / R`。
4. **传播时延：** 信号跨越距离 d，传播速度为 v，`T = d / v`。

发送时延和传播时延最容易混淆。前者像“把整列火车开出站台要多久”，取决于列车长度和出站速度；后者像“火车头从北京开到上海要多久”，取决于距离和行进速度。

典型案例：1500 字节分组到底慢在哪里

在 100 Mbit/s 链路上发送一个 1500 B 分组：

- 分组长度 `L = 1500 × 8 = 12000 bit`
- 发送时延 `12000 / 100000000 = 0.00012 s = 0.12 ms`
- 若光纤长 1000 km，传播速度近似 `2 × 10^8 m/s`，传播时延约 `5 ms`

这里传播时延远大于发送时延。把链路升级到 1 Gbit/s 只能降低发送时延，不能让光传播得更快。

1.6 带宽、吞吐量与时延带宽积

- **带宽/链路速率：** 理论上单位时间可发送多少比特。
- **吞吐量：** 应用实际得到的有效速率，会受瓶颈链路、拥塞、协议开销和对端处理能力影响。
- **时延带宽积：** 传播时延 × 带宽，表示链路上同时“在途”的比特量，像管道容积。
- **RTT：** 数据从一端到另一端，再收到回应的往返时间。
- **抖动：** 时延随时间变化，实时音视频非常敏感。

> **本章易错点：** `MB/s` 与 `Mbit/s` 相差 8 倍；发送时延不等于传播时延；带宽高不保证 RTT 小；路由器不是收到第一个比特就一定能转发完整分组，经典存储转发模型要等整包到齐。

本章过关问题

1. 为什么互联网使用分组交换仍然可能丢包？
2. 经过路由器时，帧、IP 数据报、TCP 报文段分别会发生什么变化？
3. 下载速度慢可能是带宽问题，也可能是什么其他问题？
4. 解释发送时延与传播时延的区别。

第二章 应用层：浏览器、DNS、HTTP、邮件与文件传输

2.1 应用进程怎样通信

网络应用通常采用两种架构：

- **客户端/服务器（C/S）：** 服务器长期运行并在固定或可发现的地址等待，客户端主动发起请求。网页、数据库和网盘最常见。
- **对等（P2P）：** 节点既请求资源也提供资源，可扩展性强，但节点在线状态、地址变化和管理更复杂。

应用通过 **Socket（套接字）** 使用传输层。把它理解成进程通向网络的“门”。服务器常见生命周期是：创建套接字 → 绑定本地地址和端口 → 监听 → 接受连接 → 收发数据 → 关闭。客户端通常是：创建套接字 → 连接服务器 → 收发数据 → 关闭。

一条 TCP 连接常用四元组标识：

```
(源 IP, 源端口, 目的 IP, 目的端口)
```

因此，同一台服务器的 443 端口可以同时服务成千上万客户端，因为每条连接的客户端 IP 或临时端口不同。

2.2 HTTP 报文

HTTP 是应用层请求—响应协议。一个简化请求如下：

```
GET /courses/network HTTP/1.1
Host: www.example.com
Accept: text/html
Connection: keep-alive
```

响应可能是：

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1250
Cache-Control: max-age=300

<html>...</html>
```

常见方法：

方法	主要语义	典型用途	重要提醒
GET	获取资源表示	打开页面、查询数据	参数可出现在 URL；不应被设计成修改服务器状态
POST	提交数据或触发处理	登录、创建订单	不天然等于“加密”，是否保密取决于 HTTPS
PUT	用给定表示创建或整体替换资源	REST API 更新	通常设计为幂等
PATCH	局部更新	修改部分字段	语义由 API 约定
DELETE	删除资源	删除记录	通常设计为幂等
HEAD	只取响应首部	检查资源、缓存验证	不返回响应体

状态码按首位分类：

- `2xx` 成功，如 200、201、204。
- `3xx` 重定向或缓存相关，如 301、302、304。
- `4xx` 请求侧问题，如 400、401、403、404、429。
- `5xx` 服务器或上游问题，如 500、502、503、504。

2.3 无状态、Cookie 与 Session

HTTP 本身不要求服务器记住上一条请求。网站为了保持登录状态，常用以下配合：

1. 登录成功后，服务器创建会话或签发令牌。
2. 响应通过 `Set-Cookie` 让浏览器保存标识。
3. 后续请求自动携带 `Cookie`。

4. 服务器据此查会话或校验令牌。

Cookie 是浏览器保存并按规则发送的小段数据，不等于服务器端 Session。安全属性要关注 `Secure`、`HttpOnly`、`SameSite`、有效期和作用域。

2.4 连接复用与 HTTP 版本

- **HTTP/1.0 典型非持久连接：** 每个对象可能重新建立 TCP，握手开销明显。
- **HTTP/1.1：** 默认持久连接，可在一条连接上连续请求多个对象；流水线受队头阻塞等限制，实际支持情况复杂。
- **HTTP/2：** 在一条 TCP 连接内用二进制帧多路复用多个流，并压缩首部。它缓解应用层队头阻塞，但底层 TCP 丢包仍会影响同连接中的多个流。
- **HTTP/3：** 运行在 QUIC 之上。QUIC 基于 UDP 在用户态实现可靠传输、流级多路复用，并集成 TLS，可减少连接建立等待并避免 TCP 层面的跨流队头阻塞。

> **不要把“UDP”直接等同于“不可靠”。** UDP 本身不提供可靠性，但 QUIC 可以在 UDP 之上自行实现确认、重传、拥塞控制和安全握手。

2.5 缓存、代理与 CDN

缓存的核心是“能否复用旧副本”：

- 新鲜度缓存：`Cache-Control: max-age=...`
- 条件请求：客户端带 `If-None-Match` 或 `If-Modified-Since`，未变化时服务器返回 304。
- 代理缓存：由组织或运营商中的中间节点复用内容。
- CDN：把静态或可缓存内容分布到离用户更近的边缘节点，降低 RTT 和源站压力。

缓存不是越久越好。更新频繁的数据若缓存策略错误，会出现“服务器改了，用户仍看到旧页面”。

2.6 DNS：名字怎样变成 IP

DNS 是分布式、分层的命名系统。典型解析链路是：

```
浏览器/系统缓存
  → 本地递归解析器
    → 根服务器（告诉你 .com 去哪里问）
      → 顶级域服务器（告诉你 example.com 去哪里问）
        → 权威服务器（给出最终记录）
```

客户端通常向递归解析器提出“帮我查到底”的递归请求；解析器向根、顶级域和权威服务器执行迭代查询。缓存按 TTL 保存结果，从而减少延迟和上游负载。

常见记录：

记录	含义	示例用途
A	域名到 IPv4 地址	网站 IPv4
AAAA	域名到 IPv6 地址	网站 IPv6
CNAME	别名指向规范名	CDN 或多域名管理
MX	邮件交换服务器	邮件投递
NS	域的权威服务器	委派
TXT	任意文本	域名验证、邮件反伪造策略

传统 DNS 多使用 UDP 53；响应过大、需要可靠传输或执行某些管理操作时也可使用 TCP 53。现代环境还可能使用加密 DNS，如 DoT 或 DoH。考试题若采用经典口径，应根据题设作答，不要写成“DNS 永远只用 UDP”。

2.7 电子邮件与 FTP

邮件系统包含发送端用户代理、邮件服务器和接收端用户代理：

- SMTP 用于提交与服务器间转发，经典中继端口为 25；用户提交常见 587，安全部署还可能使用 TLS。
- IMAP 强调邮件保留在服务器并支持多设备同步；POP3 更偏向下载式访问。
- MIME 让邮件能够携带不同字符集、HTML 和附件。

FTP 使用独立的控制连接和数据连接。控制连接通常连服务器 21 端口；主动模式下服务器常从 20 端口连接客户端指定端口，**被动模式则由服务器开放另一个临时端口并由客户端连接。**因此“FTP 数据连接永远是 20 端口”是过度简化。

2.8 典型案例：首次打开一个 HTTPS 页面

只看应用层，会发生这些事：

1. 浏览器解析 URL，识别协议、主机名、端口和路径。
2. 检查浏览器缓存、HSTS 等本地策略。
3. 请求 DNS 解析 `www.example.com`，可能直接命中缓存。
4. 得到一个或多个 IP 后选择地址并建立传输层连接。
5. 完成 TLS 身份验证和密钥协商。
6. 发送 HTTP 请求，收到 HTML。
7. 解析 HTML 后继续请求 CSS、JavaScript、图片和字体。
8. 复用连接、缓存和 CDN 共同决定页面速度。

本章实验

1. 浏览器按 F12 打开 Network，刷新网页，观察首个文档请求、状态码、响应头、Cookie 和缓存命中情况。
2. 执行 `nslookup www.example.com` 或 `Resolve-DnsName www.example.com`，比较 A 与 AAAA 记录。
3. Wireshark 使用 `dns`、`http`、`tls` 过滤器，分别观察可见字段与加密后的载荷。

本章过关问题

1. Cookie 与 Session 有什么区别？
2. HTTP/2 已经多路复用，为什么 HTTP/3 仍要改变底层传输？

3. 递归查询与迭代查询分别是谁替谁继续问？
4. 为什么不能说“POST 比 GET 安全”？
5. FTP 被动模式的数据连接由谁主动建立，连接哪个端口？

第三章 传输层：端口、UDP、TCP 与拥塞控制

3.1 从“送到主机”到“送到进程”

IP 地址找到主机，端口号找到进程。传输层把多个应用的数据汇集后交给网络层，称为**多路复用**；接收时根据端口和连接标识交给正确套接字，称为**多路分解**。

端口范围是 0 ~ 65535。服务器常监听约定端口，客户端通常由操作系统分配临时端口。常见端口需要识别，但工程上不能只靠端口断言协议，因为服务可以运行在任意端口。

端口	常见服务	端口	常见服务
22	SSH	53	DNS
25	SMTP 中继	67/68	DHCP 服务端/客户端
80	HTTP	443	HTTPS / HTTP/3 常用
110	POP3	143	IMAP
587	邮件提交	993	加密 IMAP

3.2 UDP：最小机制，最大自由

UDP 首部只有 8 字节：源端口、目的端口、长度、校验和各 2 字节。它保留应用写入的报文边界，不建立连接，不保证到达、顺序、去重，也不内置流量控制和拥塞控制。

UDP 适合：

- 查询短、可自行重试的 DNS。
- 实时音视频或游戏状态更新，过时数据不值得等待重传。
- 需要在用户态实现特殊传输机制的协议，例如 QUIC。
- 组播、广播或简单发现协议。

选择 UDP 不等于可以无视网络。应用若高速发送而不控制速率，同样会造成拥塞。

3.3 TCP 是字节流

TCP 向应用提供可靠、有序、双向的字节流。一次 `send(100 KB)` 不保证对端一次 `recv()` 就收到 100 KB；两次 `send()` 也可能被一次 `recv()` 合并读取。原因包括缓冲、分段、调度和网络状态。

因此应用层必须自己定义报文边界，常见方法：

- 固定长度。
- 分隔符，例如一行以 CRLF 结束。
- 长度前缀，例如前 4 字节表示后续消息长度。
- 自描述格式，例如解析完整 JSON 对象，但仍要防止半包。

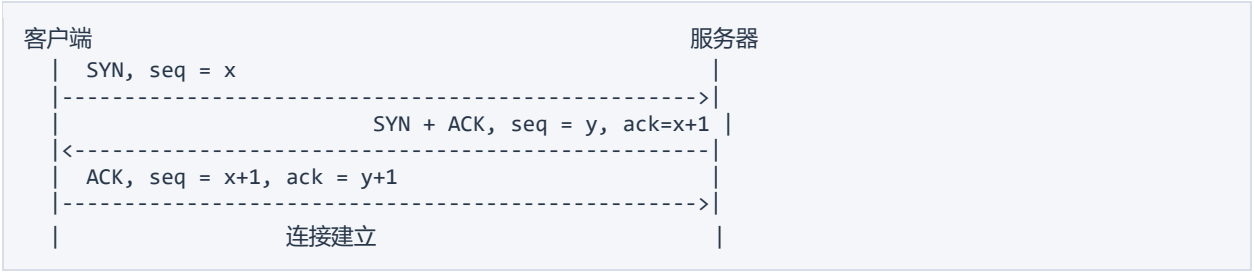
这就是所谓“粘包/半包”问题的本质：不是 TCP 出错，而是应用把字节流误当成消息队列。

3.4 TCP 首部关键字段

TCP 首部**最小 20 字节**，带选项时更长。关键字段包括：

- 源/目的端口：标识进程。
- 序号 `seq`：本报文段第一个数据字节在字节流中的编号。
- 确认号 `ack`：累计确认，表示“下一个期望收到的字节编号”。
- 数据偏移：TCP 首部长度。
- SYN、ACK、FIN、RST：建立、确认、关闭、复位连接。
- 接收窗口：流量控制。
- 校验和：覆盖 TCP 首部、数据和 IP 伪首部。
- 选项：MSS、窗口扩大、时间戳、SACK 许可等。

3.5 三次握手



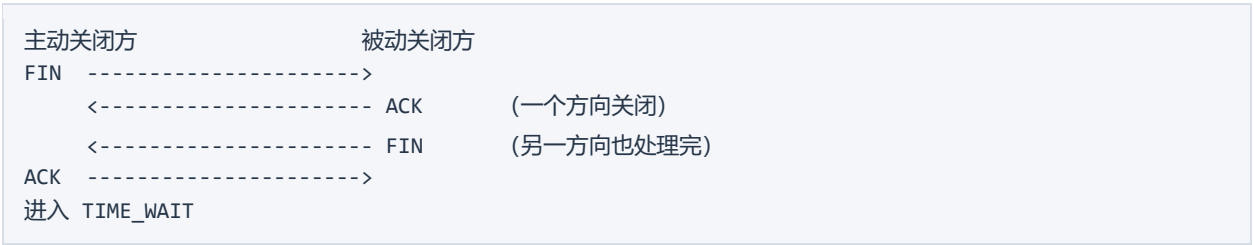
SYN 会占用一个序号，纯 ACK 通常不占用序号。三次握手至少完成：

1. 双方确认对方具备发送和接收能力。
2. 双方交换并确认初始序号，后续才能识别旧报文和新报文。
3. 服务器避免在客户端并未确认时把半开连接当成完全建立。

“防止旧连接请求”是常见考试答案，但更完整的理解是：可靠地同步双方初始状态，需要让双方都知道自己的发送已被对方确认。

3.6 四次挥手与 TIME_WAIT

TCP 是全双工，两方向可以独立关闭：



ACK 与 FIN 有时能合并，所以抓包中不一定机械地看到四个独立报文。主动关闭方等待 TIME_WAIT，主要为了：

- 最后 ACK 丢失时，还能再次响应对方重传的 FIN。
- 让旧连接中的迟到报文在网络中消失，避免污染相同四元组的新连接。

3.7 可靠传输怎样实现

TCP 的可靠性不是“链路永不丢包”，而是端系统把不可靠网络包装成可靠字节流：

- 校验和发现比特错误。
- 序号发现乱序、重复和缺口。
- 累计 ACK 表示连续收到哪里。
- 超时重传处理长时间没有确认。
- 快速重传利用重复 ACK 尽早重发疑似丢失段。
- 接收端缓存乱序段；SACK 可告诉发送端已收到哪些非连续区间。
- RTO 根据测得 RTT 动态估计，不能固定成一个适合所有网络的数。

MSS 是 TCP 单个报文段希望承载的最大应用数据量；它通常根据路径接口 MTU 推导。常见以太网 MTU 为 1500 B 时，经典 IPv4 + TCP 无选项场景的 MSS 为 `1500 - 20 - 20 = 1460 B`。

3.8 流量控制：别把接收方淹没

接收方通过 `rwnd` 通告剩余缓冲空间。发送方未确认的在途数据不能超过接收方允许范围。如果窗口降为 0，发送方不能永远沉默，否则窗口更新丢失会导致死锁；TCP 可通过持续计时器和窗口探测恢复。

流量控制关心的是**接收方能力**，不是全网是否拥堵。

3.9 拥塞控制：别把网络压垮

发送端还维护拥塞窗口 `cwnd`，综合限制可近似写成：

```
可发送的未确认数据量 ≤ min(rwnd, cwnd)
```

经典教材以 TCP Reno 解释四个机制：

- **慢启动：** `cwnd` 从较小值开始，每收到确认便增长，按 RTT 观察近似指数增加。
- **拥塞避免：** 达到 `ssthresh` 后改为近似线性增加。

- **快重传：** 收到 3 个重复 ACK 时，不等 RTO 就重传缺失段。
- **快恢复：** 对重复 ACK 指示的轻度拥塞减小窗口，但通常不像超时那样完全回到最小值。

发生超时通常意味着更严重的拥塞，经典题型中会令 `ssthresh = 原 cwnd / 2`，并把 `cwnd` 重置为 1 MSS；3 个重复 ACK 则按题目指定的 Reno 规则进入快恢复。

> **考试模型与真实实现要分开：** 现代系统的初始窗口、CUBIC/BBR 等算法以及精确恢复过程可能不同。做题严格服从题设；理解工程时，不要把“cwnd 永远从 1 开始、永远每 RTT 加 1”当成互联网唯一实现。

3.10 典型案例：数据库服务器收到半条 SQL 请求

假设协议定义“4 字节大端长度 + 消息体”。客户端发送一条 100 KB 请求：

1. TCP 把字节流分成多个报文段。
2. 服务器第一次 `recv()` 可能只得到 2 字节，连长度字段都不完整。
3. 第二次可能得到剩余 2 字节和部分消息体。
4. 服务器缓冲区先解析长度，再等待累计字节数达到完整长度。
5. 若一次读到完整消息后还有下一条消息的开头，解析器要保留剩余字节继续处理。

正确代码要循环读取、处理短读、连接关闭、超时和恶意超大长度。把“一次 send 对应一次 recv”写进协议，会产生间歇性且难复现的故障。

本章实验

1. Wireshark 过滤 `tcp.flags.syn == 1`，找到 SYN、SYN+ACK 与最终 ACK。
2. 右键一个 TCP 包选择 Follow TCP Stream，观察应用字节流如何由多个段组成。
3. 使用 `tcp.analysis.retransmission` 和 `tcp.analysis.duplicate_ack` 查找重传与重复确认。
4. 在本机查看监听端口和连接四元组，解释为什么同一个服务端口能有多条连接。

本章过关问题

1. TCP 为什么不能保留应用的消息边界？应用怎样补上边界？
2. 为什么握手是三次，而关闭通常需要四次？
3. `seq=1000` 的段携带 500 B 数据，对方正确收到后应确认多少？
4. `rwnd` 与 `cwnd` 各自防止谁出问题？
5. 看到 3 个重复 ACK 与看到 RTO 超时，经典 Reno 为什么采取不同幅度的降速？

第四章 网络层：IP、子网、转发、NAT 与 IPv6

4.1 网络层的两个动作

- **转发（数据平面）**： 对一个已经到来的分组查表，决定从哪个接口送出。
- **路由（控制平面）**： 计算或学习整张转发表应该是什么。

路由像导航规划，转发像司机到每个路口依照导航选择出口。

4.2 IPv4 地址与 CIDR

IPv4 地址 32 位，CIDR 写作 `地址/前缀长度`。前 n 位是网络前缀，剩余位用于该网络内部的地址。子网掩码就是前 n 位为 1、其余为 0。

分类编址 A/B/C 是历史方案，仍可能出现在考试中；现代网络主要使用 CIDR 和最长前缀匹配。常用特殊范围：

- `10.0.0.0/8`、`172.16.0.0/12`、`192.168.0.0/16`：私有地址。
- `127.0.0.0/8`：环回。
- `169.254.0.0/16`：IPv4 链路本地地址，常见于自动配置失败。
- `0.0.0.0`：依语境可表示未指定地址或默认路由前缀的一部分。
- `255.255.255.255`：受限广播。

传统 IPv4 子网中，主机位全 0 是网络地址，主机位全 1 是定向广播地址，普通主机不可使用。点到点链路等场景存在特殊规则，基础题通常按 `2^主机位 - 2` 计算。

4.3 固定步骤做子网题

以 `192.168.10.130/26` 为例：

1. `/26` 表示 26 位网络位，掩码为 `255.255.255.192`。
2. 最后一个八位组的块大小是 `256 - 192 = 64`。

3. 子网区间为 0 ~ 63、64 ~ 127、128 ~ 191、192 ~ 255。
4. 130 落在 128 ~ 191。
5. 网络地址：`192.168.10.128`。
6. 广播地址：`192.168.10.191`。
7. 可用地址：`192.168.10.129` ~ `192.168.10.190`。
8. 可用主机数：`2^6 - 2 = 62`。

VLSM 案例：给不同部门分地址

单位得到 `192.168.1.0/24`，需要 100、50、20 台主机的三个子网。按“先大后小”分配：

部门需求	至少地址块	前缀	网络	可用范围	广播
100 台	128 个地址	/25	192.168.1.0	.1 ~ .126	.127
50 台	64 个地址	/26	192.168.1.128	.129 ~ .190	.191
20 台	32 个地址	/27	192.168.1.192	.193 ~ .222	.223

剩余 `.224/27` 可保留。先分小网很容易造成地址碎片，导致后面的大网放不下。

4.4 IPv4 数据报首部

IPv4 首部最小 20 B。重点字段：

- 版本与首部长度。
- 总长度：首部加数据。
- 标识、DF、MF、片偏移：分片与重组。
- TTL：每经过路由器至少减 1，降为 0 时丢弃，防止无限循环。
- 协议：指示上层，如 ICMP、TCP、UDP。
- 首部校验和：只校验 IPv4 首部，因此每跳修改 TTL 后需要重算。
- 源/目的 IPv4 地址。

4.5 分片计算案例

一个总长 4000 B 的 IPv4 数据报，首部 20 B，要通过 MTU=1500 B 的链路。原始数据为 3980 B。每个非最后分片的数据长度应为不超过 1480 且是 8 的倍数：

分片	数据长度	总长度	片偏移字段	MF
1	1480	1500	0	1
2	1480	1500	185	1
3	1020	1040	370	0

片偏移的单位是 8 B，所以第二片是 `1480/8=185`，第三片是 `(1480+1480)/8=370`。每片都有自己的 IP 首部。IPv4 通常只在最终目的主机重组；分片丢一片会影响整个原始数据报。工程上常通过路径 MTU 发现与合适 MSS 尽量避免分片。

IPv6 路由器不执行这种逐跳分片；需要时由源节点使用分片扩展首部。

4.6 最长前缀匹配

若路由表同时有：

```
10.0.0.0/8       → 接口 A
10.1.0.0/16      → 接口 B
10.1.2.0/24      → 接口 C
0.0.0.0/0        → 默认网关
```

目的 `10.1.2.99` 同时匹配前三条，但 `/24` 最具体，所以选接口 C。最长前缀匹配不是选“数值最接近”的地址，也不是简单选第一条。

4.7 DHCP：自动获得网络参数

经典 DORA 流程：

1. Discover：客户端广播寻找 DHCP 服务器。
2. Offer：服务器提供地址、掩码、网关、DNS 和租期等。
3. Request：客户端选择并请求某个报价。

4. ACK：服务器确认租约。

DHCP 使用 UDP，服务端常用 67，客户端常用 68。刚启动的客户端还没有可用 IP，因此流程大量使用广播；跨网段部署时可由 DHCP Relay 转发。

4.8 NAT/PAT：许多内网主机共用公网地址

家庭路由器常执行端口地址转换：

```
192.168.1.10:51000  ┐
192.168.1.20:51000  ├─> 203.0.113.8:40001 / :40002 -> Internet
192.168.1.30:62000  ┘
```

设备维护映射表并改写 IP、端口与相关校验和。返回报文根据公网端口找到内部主机。NAT 缓解 IPv4 地址不足，也天然阻碍外部主动连接；它并不等于完整防火墙，安全仍依赖状态策略和主机防护。

4.9 ICMP、ping 与 traceroute

ICMP 传递差错和控制信息。`ping` 常使用 Echo Request/Reply 判断可达性和 RTT，但目标或中间设备可能屏蔽 ICMP，所以“ping 不通”不等于业务必定不通。

`traceroute/tracert` 让探测报文的 TTL 从 1 递增。第一个路由器把 TTL 减到 0 并返回 ICMP 超时，第二次 TTL=2 暴露第二跳，如此逐步得到路径。某一跳显示星号，可能只是路由器不回应或回包被限速，不必然意味着后续路径中断。

4.10 IPv6

IPv6 地址 128 位，使用冒号十六进制；一段连续全 0 可以用 `::` 压缩一次，每组前导 0 可省略。基础特性：

- 基本首部固定 40 B，字段更精简，附加功能放在扩展首部。
- 地址空间巨大，支持更合理的层次化分配。
- 不使用广播，更多依靠组播和邻居发现。
- 邻居发现使用 ICMPv6，承担类似 ARP、路由器发现等功能。

- 过渡方式包括双栈、隧道和协议转换。

> **本章易错点：** IPv4 首部是“最小 20 B”而非永远 20 B；片偏移要除以 8；最长前缀匹配选最具体规则；NAT 场景中 IP/端口并非端到端不变；ping 只能证明特定 ICMP 往返结果。

本章过关问题

1. `172.16.35.123/20` 的掩码、网络地址和广播地址是什么？
2. 为什么跨网段通信时主机并不直接 ARP 查询远端主机的 MAC？
3. 分片为何会放大丢包代价？
4. 最长前缀匹配与默认路由怎样配合？
5. NAT 为什么让外网主动访问内网服务变困难？

第五章 路由：数据报究竟走哪条路

5.1 路由协议要解决什么

每台路由器只能把分组交给下一跳，但全网需要逐步形成一致、可用的路径。路由算法要面对：拓扑变化、链路代价、环路、收敛速度、扩展性和管理策略。

互联网被划分成许多自治系统（AS）：运营商、大学、云厂商可各自管理内部路由，同时用域间协议交换可达性。

5.2 距离向量与链路状态

维度	距离向量（DV）	链路状态（LS）
节点掌握的信息	邻居告诉我的“到各网距离”	全区域拓扑与链路代价
主要计算思想	Bellman-Ford 迭代	Dijkstra 最短路
信息交换	周期或触发地与邻居交换向量	泛洪链路状态通告
优点	思路简单、状态较少	收敛通常更快、路径计算清晰
典型问题	坏消息传得慢、无穷计数	泛洪和拓扑数据库开销

DV 的核心公式可以口语化为：“我到目的地的最短距离 = 在所有邻居中，选 到邻居的代价 + 邻居声称的距离 最小者。”

LS 则先获得地图，再从自己出发逐轮确定最近的未确定节点。

5.3 Dijkstra 小案例

拓扑代价：A-B=2，A-C=5，B-C=1，B-D=4，C-D=1。

1. 从 A 开始：暂定 B=2、C=5、D=∞。
2. 选最小 B=2，借 B 更新：C=min(5,2+1)=3，D=min(∞,2+4)=6。
3. 选 C=3，借 C 更新：D=min(6,3+1)=4。

4. 选 D=4，完成。

A 到 D 的最短路径是 A→B→C→D，总代价 4。做题时必须记录“已确定集合、各节点当前距离、前驱”，不能只写最终答案。

5.4 RIP、OSPF 与 BGP

- **RIP：** 经典距离向量 IGP，以跳数为度量，15 跳以内可达，16 表示不可达，适合较小网络。
- **OSPF：** 链路状态 IGP，在区域内传播链路状态并运行最短路径计算，可按区域分层扩展。
- **BGP：** AS 之间的路径向量协议，通告网络前缀和 AS 路径等属性。它优先满足策略与商业关系，不保证地理距离或跳数最短。

典型案例：为什么“更短”的国际路径没被选中

某运营商到目标前缀有两条路：A 路只有 3 个 AS，但要经过昂贵的上游；B 路有 5 个 AS，却经过免费对等互联。BGP 可能根据本地策略优先 B。互联网路由不是纯数学最短路，而是“可达性 + 策略 + 稳定性”的结果。

5.5 路由环路与收敛

拓扑变化后，各路由器不会瞬间同时知道真相。在收敛期间，错误信息可能造成环路、黑洞或绕路。常见缓解思想包括：序列号、触发更新、毒性逆转、最大度量、链路状态通告老化等。不同协议机制不同，但目标一致：让旧信息尽快失效并阻止谣言循环放大。

本章过关问题

1. 转发表和路由表有什么关系？
2. 为什么 DV 容易出现“坏消息传得慢”？
3. OSPF 和 BGP 的管理范围与目标有何不同？
4. 为什么 BGP 不一定选 AS 数量最少的路径？

第六章 数据链路层：以太网、交换机、ARP 与 VLAN

6.1 链路层只负责一跳

IP 负责端到端跨网转发，链路层负责在一条具体链路上把数据报交给下一节点。每换一条链路，帧格式、源/目的 MAC 乃至最大帧大小都可能变化。

主要任务包括：成帧、差错检测、介质访问控制，有些链路还提供重传和流量控制。

6.2 以太网帧

经典以太网帧结构：

```
前导码/SFD | 目的 MAC 6B | 源 MAC 6B | 类型/长度 2B | 数据 46~1500B | FCS 4B
```

从目的 MAC 到 FCS，经典帧长通常为 64 ~ 1518 B；802.1Q VLAN 标签会增加 4 B。前导码用于同步，通常不计入所说的以太网帧长度。

MAC 地址为 48 位。它可以是全球管理地址，也可以是本地管理地址；系统还可以随机化或修改 MAC，因此“MAC 永远全球唯一且绝不变化”不严谨。

6.3 交换机自学习

交换机收到帧后：

1. 学习**源 MAC → 入端口**，并设置老化时间。
2. 查目的 MAC：

- 已知且出口不同：只转发到对应端口。
- 已知且出口就是入端口：过滤，不再转发。
- 未知单播：在同一 VLAN 内向除入端口外的其他端口**泛洪**。
- 广播：在同一 VLAN 内泛洪。

“泛洪”比“广播”更准确：未知单播帧的目的 MAC 仍是某个单播地址，只是交换机暂时不知道它在哪。

自学习案例

空表交换机有端口 1:A、2:B、3:C。

- A→B：学习 A→1；B 未知，向 2、3 泛洪。
- B→A：学习 B→2；A 已知，只发端口 1。
- C→A：学习 C→3；A 已知，只发端口 1。

此时表里已有 A、B、C，后续单播大多不再打扰无关端口。

6.4 ARP：下一跳 IP 对应哪个 MAC

IPv4 主机发送以太网帧前，需要知道下一跳 MAC：

- **同一子网：** 下一跳就是目标主机，ARP 查询目标 IP。
- **不同子网：** 下一跳是默认网关，ARP 查询网关 IP，而不是远端主机 IP。

ARP Request 通常广播：“谁拥有这个 IPv4 地址？”目标以 ARP Reply 单播回应，主机将映射缓存一段时间。ARP 位于 IP 与具体链路交界处，不同教材可能归类不同；比背层号更重要的是理解它只解析**本地链路上的下一跳。**

6.5 CRC 差错检测

发送方把比特串视为多项式，用生成多项式做模 2 除法，把余数作为 FCS。模 2 加减都是异或，不进位、不借位。

例：数据 `101110`，生成多项式比特串 `1011`（3 阶）：

1. 数据后补 3 个 0：`101110000`。
2. 用 `1011` 做模 2 长除，余数为 `110`。
3. 发送码字：`101110110`。

4. 接收方再除以 `1011`，余数为 0 表示“未检测到错误”。

CRC 能高概率检错，但不是加密，也通常不负责定位并纠正错误。若需要纠错，可使用海明码、前向纠错或上层重传机制。

6.6 共享介质与 CSMA/CD

传统半双工共享以太网使用 CSMA/CD：先侦听，空闲再发；发送过程中检测碰撞；碰撞后停止并按二进制指数退避随机等待。

为什么存在最小帧长？最坏情况下，远端碰撞信号要在一个往返传播时间 `2τ` 内返回。发送者必须仍在发送，才会意识到碰撞，所以：

```
最小帧发送时间 ≥ 2τ
最小帧长 ≥ 2τ × 链路速率
```

现代交换式全双工以太网的每条链路不再共享冲突域，不使用 CSMA/CD 竞争，但该原理仍是经典考试重点。

6.7 VLAN 与 Trunk

VLAN 在一套交换基础设施上划分多个逻辑广播域。不同 VLAN 的主机即使连接同一交换机，也需要三层设备路由才能互通。

- Access 端口通常承载一个 VLAN，连接终端。
- Trunk 端口承载多个 VLAN，常用 802.1Q 标签区分。
- VLAN 提供广播域隔离与管理便利，但不是完整的安全边界；仍需 ACL、防火墙和正确配置。

6.8 地址在跨网通信中怎样变化

主机 A 访问跨网段服务器 B，第一跳：

```
以太网源 MAC = A 的 MAC
以太网目的 MAC = 默认网关的 MAC
IP 源地址 = A 的 IP
IP 目的地址 = B 的 IP
TCP 源/目的端口 = 客户端临时端口 / 服务端口
```

路由器转发到下一链路时重新生成链路层首尾，MAC 改为本接口与下一跳的 MAC；IP 的 TTL 会改变。若路径含 NAT，IP 和端口也可能被改写。

本章实验

1. 查看本机 ARP/邻居缓存，解释网关条目。
2. Wireshark 过滤 `arp`，观察 Request 广播与 Reply。
3. 过滤 `eth.addr == xx:xx:xx:xx:xx:xx`，跟踪某个 MAC 的帧。
4. 画出“电脑—家用路由器—运营商”前两跳，并标注每一跳的 MAC、IP 和端口。

本章过关问题

1. 交换机学习源 MAC 还是目的 MAC？为什么？
2. 未知单播泛洪与广播帧有什么区别？
3. A 跨网段访问 B 时，第一帧的目的 MAC 为什么不是 B？
4. 全双工交换式以太网为什么不再需要 CSMA/CD？
5. VLAN 隔离的是什么范围？不同 VLAN 怎样通信？

第七章 物理层与无线网络：比特如何穿过真实世界

7.1 数字数据怎样变成信号

物理层规定接口、信号、电平、时钟、编码、调制和介质。比特率表示每秒传多少比特；波特率表示每秒发生多少次符号变化。一个符号若能表示多个比特，二者并不相等。

常见基带编码：

- NRZ：实现简单，但长串相同电平不利于时钟同步。
- 曼彻斯特：每个比特中间有跳变，自带同步，但需要更高符号变化率。
- 差分曼彻斯特：利用是否发生特定跳变表达数据，对极性反转更稳健。

7.2 奈奎斯特与香农

无噪声理想低通信道的码元速率受奈奎斯特限制：

```
C = 2W log2(V)
```

W 是带宽 Hz，V 是一个符号可取的离散电平数。有噪声信道的理论容量受香农公式限制：

```
C = W log2(1 + S/N)
```

若给的是分贝：`SNR(dB) = 10 log10(S/N)`，先转换成普通比值。

计算案例

带宽 3 kHz，16 个信号电平，信噪比 30 dB：

- 奈奎斯特：`2 × 3000 × log2(16) = 24 kbit/s`。
- `30 dB` 对应 `S/N = 1000`。
- 香农：`3000 × log2(1001) ≈ 29.9 kbit/s`。
- 两个上限都要满足，因此理论上限取较小的约 24 kbit/s。

7.3 复用

- FDM：不同用户使用不同频带。
- TDM：不同用户轮流使用时间片。
- WDM：光纤中用不同波长并行传输，本质类似光的频分。
- CDM：用户共享时间和频率，用相互可区分的码片序列分离。

复用解决“多路信号怎样共享介质”，多路复用/分解也会出现在传输层，但两者处理的对象和层次不同。

7.4 Wi-Fi 架构

基础设施模式中，终端关联到 AP，多个 AP 可通过分布系统接入同一网络。常见概念：

- SSID：网络名称，不等于安全凭证。
- BSS：一个基本服务集合。
- 信道：相邻 AP 若信道重叠会相互干扰。
- Beacon：AP 周期广播能力信息；终端也可主动探测。

7.5 CSMA/CA

无线设备发送时自己的信号远强于远端信号，难以像有线共享以太网那样边发边可靠检测碰撞；隐藏终端也可能彼此听不到。Wi-Fi 主要采用碰撞避免：

1. 侦听信道。
2. 空闲一段规定时间后进入随机退避。
3. 计数到 0 才发送；忙时冻结计数。
4. 接收方以链路层 ACK 确认。
5. 未收到 ACK 则扩大竞争窗口后重试。

RTS/CTS 可以预约信道并缓解隐藏终端，但有额外开销，并非每个数据帧都必须使用。

7.6 无线典型问题

- **隐藏终端：** A 与 C 都能到 B，却彼此听不到，于是可能同时向 B 发送。
- **暴露终端：** 节点误以为附近发送一定会干扰自己的目标，白白不敢发送。
- **同频干扰：** 多个 AP 或设备争用同一信道，信号满格也可能很慢。
- **漫游：** 终端从一个 AP 重新关联到另一个 AP；认证、地址保持和上层重传共同影响是否“无感”。

典型案例：Wi-Fi 满格但视频卡

“满格”主要反映接收信号强度，不代表信道空闲、干扰小或上游宽带充足。排查时应区分：无线重传率、信道占用、AP 到路由器的回程、运营商链路、DNS、视频 CDN 和终端性能。

本章过关问题

1. 比特率与波特率什么时候不同？
2. 为什么实际信道容量要同时受奈奎斯特与香农限制？
3. Wi-Fi 为什么用 CA，而传统共享以太网可用 CD？
4. RTS/CTS 主要缓解什么问题，为什么不总是开启？

第八章 网络安全：HTTPS 保护了什么

8.1 三个安全目标

- **机密性：** 未授权者看不懂内容。
- **完整性：** 篡改能够被发现。
- **身份认证：** 确认对端是谁。

可用性同样重要：系统在攻击或故障下仍能提供服务。不同机制解决不同目标，单独“加密”不能自动提供全部安全性。

8.2 对称加密、非对称密码与哈希

机制	解决什么	特点	典型用途
对称加密	机密性	同一密钥加解密，速度快	大量业务数据加密
公钥密码	密钥协商、身份相关操作	公私钥成对，运算较慢	建立共享秘密、数字签名
哈希	数据指纹	单向、定长、雪崩效应	完整性结构、密码存储组件
MAC/HMAC	带共享密钥的完整性与认证	只有持钥者能生成有效标签	API 请求认证、协议完整性
数字签名	身份与完整性、可验证性	私钥签名，公钥验证	证书、软件签名

“用私钥加密就是签名”是方便入门但不够严谨的说法。真实签名算法有专门的编码、哈希和安全要求。

8.3 证书与信任链

服务器证书把域名、公钥、有效期等信息绑定在一起，并由 CA 签名。浏览器验证：

1. 证书中的域名是否匹配当前网站。
2. 当前时间是否在有效期内。
3. 签名能否沿中间 CA 链到系统信任的根 CA。

4. 证书用途、算法和策略是否可接受。

证书不是“加密数据本身的钥匙证明书”这么简单；它最关键的作用是让客户端有依据相信自己拿到的公钥确实属于目标域名。

8.4 TLS/HTTPS 的高层流程

以常见 TLS 1.3 思想为例：

1. 客户端与服务器协商协议参数并发送密钥交换材料。
2. 服务器发送证书及对握手内容的签名证明。
3. 客户端验证证书和签名。
4. 双方基于密钥交换结果导出会话密钥。
5. 后续 HTTP 数据使用高效的对称认证加密保护。

HTTP/1.1 和 HTTP/2 通常先建立 TCP，再进行 TLS；HTTP/3 的 QUIC 将传输与 TLS 握手紧密结合。

HTTPS 通常保护 HTTP 路径、首部、Cookie 和正文不被链路中间人直接读取或篡改，但仍会暴露一些连接元数据，如对端 IP、流量大小与时间特征。DNS 是否加密取决于所用解析方式。

8.5 常见攻击与防护思路

- **窃听：** 用 TLS、VPN 或安全链路保护内容。
- **中间人攻击：** 严格证书验证、不要无视证书警告。
- **ARP 欺骗：** 局域网攻击者伪造映射；可通过交换机安全特性、静态绑定、加密上层协议缓解。
- **DNS 污染/劫持：** 使用可信解析器、DNSSEC 验证或加密 DNS，具体机制解决的目标不同。
- **SYN Flood / DDoS：** 限速、SYN Cookie、负载清洗、分布式防护与容量规划。
- **扫描与暴露服务：** 最小化开放端口，使用防火墙、鉴权、补丁和日志监控。
- **应用层漏洞：** HTTPS 无法修复 SQL 注入、越权、弱密码或恶意服务器。

典型案例：浏览器提示证书不受信任

正确反应不是“反正有小锁就点继续”。可能原因包括系统时间错误、证书过期、域名不匹配、企业代理解密、公共 Wi-Fi 劫持或真实攻击。先核对域名和时间，在可信网络重试，检查证书颁发者；涉及账号、支付或敏感数据时不要绕过警告。

本章过关问题

1. 对称加密很快，为什么还需要公钥密码？
2. 数字签名与 HMAC 都能检查完整性，它们的信任前提有何不同？
3. HTTPS 能否隐藏你正在连接的服务器 IP？能否阻止服务器自身作恶？
4. 证书链验证为什么能降低中间人攻击风险？

第九章 五个综合案例：把各层串成一条链

9.1 案例一：浏览器输入网址到页面显示

这是全书“毕业题”。可以按 18 步复述：

1. 浏览器解析 URL，确定协议、域名、端口和路径。
2. 检查缓存、代理、HSTS 和已有连接。
3. 系统查询 DNS 缓存；未命中则向递归解析器查询。
4. 解析器经缓存或根/TLD/权威链得到 A/AAAA 记录。
5. 主机根据目的 IP 与掩码判断目标是否同网段。
6. 若跨网段，查 ARP/邻居缓存获得默认网关的链路层地址。
7. 生成帧，经 Wi-Fi/AP/交换机交给网关。
8. 家用网关可能进行 NAT/PAT。
9. 各路由器按最长前缀匹配逐跳转发，TTL/Hop Limit 递减。
10. 若使用 TCP，客户端与服务器三次握手并协商 MSS、窗口扩大等选项。
11. TLS 验证证书、协商密钥并确认握手完整性。
12. 浏览器发送 HTTP 请求。
13. 服务器或 CDN 处理请求，可能查询缓存、应用和数据库。
14. 响应被分成传输层数据单元和 IP 数据报，经不同链路封装返回。
15. TCP/QUIC 处理乱序、丢包、流量和拥塞；TLS 解密并验证数据。
16. 浏览器解析 HTML，构建 DOM，并发现更多资源。
17. 复用连接并行请求 CSS、JavaScript、图片和字体。
18. 浏览器完成样式计算、布局、绘制与交互；缓存策略影响下次访问。

复述时必须指出：MAC 逐跳变，IP 通常端到端保持但 NAT 可改写，端口标识进程，域名只在应用/DNS 语境中有意义。

9.2 案例二：在线视频会议为什么宁可偶尔糊也不要停

实时通话最怕等待。某一帧视频晚到 2 秒，即使最终完整，也已经失去播放价值。因此系统常采用：

- UDP 或基于 UDP 的实时传输，应用自行处理序号、时间戳、抖动缓冲。
- 关键帧、前向纠错、选择性重传和码率自适应。
- 根据丢包、RTT 和抖动降低分辨率，而不是无限重传。
- 音频优先，因为人对声音断裂通常比画质下降更敏感。

这里的“可靠”不再等于每个字节最终到达，而是用户在正确时间获得可理解的媒体。

9.3 案例三：在线游戏为什么常用 UDP，但登录仍可能用 TCP/HTTPS

位置更新频繁，旧坐标被新坐标覆盖，重传旧包可能让角色“倒退”。游戏可用 UDP 发送高频状态，同时自己为关键事件做确认。账号登录、支付、背包变更更强调一致性和安全，适合 HTTPS/TCP 或在可靠自定义通道上处理。

一个应用可以同时使用多种传输策略，不必在“全 TCP”与“全 UDP”之间二选一。

9.4 案例四：数据库服务偶发卡死

现象：客户端能连接，但大请求偶发超时。分层排查：

1. 应用层：请求长度字段是否解析正确？服务线程是否被慢查询占满？
2. 传输层：是否错误假设一次 recv 得到完整请求？发送缓冲是否阻塞？是否大量重传或零窗口？
3. 网络层：路由是否抖动？MTU 黑洞是否让大包失败而小包正常？
4. 链路层：网卡错误、交换机端口丢包、Wi-Fi 重传是否异常？

5. 物理层：线缆、光模块、信号质量是否有错误计数？

抓包若显示服务器持续通告零窗口，重点是接收应用不够快；若连续重传没有 ACK，重点转向路径丢包、防火墙或对端；若三次握手后服务器立刻 RST，可能没有正确服务进程或应用拒绝连接。

9.5 案例五：能 ping 通，但网页打不开

`ping` 只验证特定 ICMP 往返，不验证 DNS、TCP 443、TLS 和 HTTP。建议从低到高定位：

1. IP 配置：地址、掩码、网关是否正确。
2. 本地链路：能否到达网关，ARP 是否正常。
3. 路由：能否到目标 IP，路径是否异常。
4. DNS：域名能否解析，解析结果是否正确。
5. 传输：目标端口能否建立连接，是否被防火墙拦截。
6. TLS：证书、时间、协议协商是否正常。
7. HTTP/应用：状态码、代理、服务端错误和浏览器缓存。

> **排障原则：** 一次只验证一个假设；记录时间、地址、端口和错误信息；“通/不通”要说明通的是哪一层、哪种协议、哪个方向。

综合口述任务

不看资料，连续讲 10～15 分钟“网页之旅”，至少出现这些词：DNS 缓存、递归解析、ARP、默认网关、帧、NAT、最长前缀匹配、TCP/QUIC、TLS 证书、HTTP、CDN、拥塞控制、浏览器渲染。

第十章 Wireshark 实验手册

10.1 抓包前的基本意识

- 只在自己有权管理或明确获准的设备和网络上抓包。
- HTTPS 载荷加密是正常现象，不代表“没有抓到”。
- 抓包位置决定你能看到什么：本机抓包能看到本机流量，交换网络中通常看不到其他端口的单播。
- 操作系统卸载功能可能让本机抓包显示“校验和错误”或超大段，这是抓包点与网卡硬件分工造成的假象。
- 先记录复现步骤和时间，再抓包；过滤范围越明确，分析越快。

10.2 常用显示过滤器

目标	过滤器
DNS	`dns`
ARP	`arp`
ICMP/IPv6 ICMP	`icmp or icmpv6`
某个 IP	`ip.addr == 192.0.2.10`
某个 TCP 端口	`tcp.port == 443`
SYN	`tcp.flags.syn == 1`
初始 SYN，不含 ACK	`tcp.flags.syn == 1 and tcp.flags.ack == 0`
TCP 重传	`tcp.analysis.retransmission`
重复 ACK	`tcp.analysis.duplicate_ack`
HTTP	`http`
TLS 握手	`tls.handshake`
指定会话	`tcp.stream eq 0`

捕获过滤器和显示过滤器语法不同。初学时可以先正常捕获，再用显示过滤器缩小范围，避免一开始过滤过严导致关键包没抓到。

10.3 实验一：DNS + 页面访问

1. 清理或避开缓存，开始抓包。
2. 打开一个此前未访问的域名。
3. 用 `dns` 找 Query/Response，记录事务 ID、查询名、记录类型、响应地址和 TTL。
4. 找到随后连接该 IP 的 TCP 或 QUIC 流量。
5. 对照浏览器 Network 面板，确认域名、远端地址和请求时间线。

10.4 实验二：TCP 状态机

1. 选择一个新的 TCP 连接。
2. 找三次握手，记录两端初始序号、MSS、窗口扩大和 SACK 许可。
3. 选择 Follow TCP Stream，观察应用数据。
4. 找 FIN/ACK 或 RST，判断正常关闭还是异常复位。
5. 注意 Wireshark 默认常显示相对序号，便于阅读但不是线上原始数值。

10.5 实验三：ARP 与第一跳

1. 查看默认网关 IP。
2. 清除测试条目或等待缓存失效后开始抓包。
3. 访问跨网段地址。
4. 观察主机查询的是网关 MAC，而不是远端服务器 MAC。
5. 在后续数据帧中验证目的 MAC 与 ARP Reply 一致。

10.6 实验四：重传与性能

出现卡顿时：

1. 过滤重传、Fast Retransmission、Duplicate ACK 和 Zero Window。
2. 查看 RTT 图、吞吐图和时间序列。
3. 区分“网络丢包”“接收方处理慢”“应用迟迟不发数据”。
4. 不要仅凭一个被标记的重传下结论；结合方向、序号、ACK、时间和抓包位置。

10.7 实验报告模板

每次实验只需一页：

1. 目标：要验证哪个机制。
2. 环境：时间、设备、接口、地址，可对敏感地址做脱敏。
3. 操作：怎样触发流量。
4. 证据：关键帧编号、字段和时序图。
5. 解释：证据如何支持结论。
6. 异常与反思：哪些现象与预期不同，可能原因是什么。

第十一章 高频计算题与解题模板

11.1 子网划分

固定步骤：

1. 写出前缀或掩码，确定主机位 h。
2. 用块大小或按位与求网络地址。
3. 广播地址 = 下一子网网络地址 - 1。
4. 普通可用范围在二者之间。
5. 验证需求：`2^h - 2` 是否够用。
6. VLSM 先分最大需求，并检查边界对齐。

11.2 存储转发与流水线

有 k 条同速链路，每条发送时延 `L/R`，每条传播时延 `d`，忽略排队与处理：

- 一个分组端到端：`k × L/R + k × d`。
- N 个等长分组流水发送，最后一包到达：`(N + k - 1) × L/R + k × d`。

案例：3 条链路，`L=12000 bit`，`R=2 Mbit/s`，每条传播 5 ms：一个分组发送时延 6 ms，端到端 `3×6 + 3×5 = 33 ms`。若有 4 个分组流水，最后一个到达约 `(4+3-1)×6 + 15 = 51 ms`。

11.3 IPv4 分片

1. 原始数据长度 = 总长度 - 原首部长度。
2. 每片最大数据 = `floor((MTU - 新首部长度)/8) × 8`。
3. 逐片填写总长度。
4. 片偏移 = 该片数据在原数据中的起点 / 8。
5. 除最后一片外 MF=1，最后一片 MF=0；DF=1 时路由器不能分片。

11.4 CRC

1. 生成多项式最高次数为 r，数据后补 r 个 0。
2. 用生成比特串模 2 长除，只做异或。
3. r 位余数补到数据后形成码字。
4. 接收端同除，余数非 0 表示检测到错误。

11.5 滑动窗口

- 停止等待：发送窗口 1，简单但长肥管道利用率低。
- GBN：接收窗口通常 1，累计确认；丢失后可能回退重传该帧及后续帧。n 位序号时经典条件 `W_s ≤ 2^n - 1`。
- SR：接收方缓存乱序帧，只重传缺失帧。为避免新旧序号混淆，经典对称条件通常 `W_s, W_r ≤ 2^(n-1)`。

做题先写清：序号空间、发送/接收窗口、ACK 是累计还是选择性、计时器按帧还是按最早未确认帧。

11.6 TCP 拥塞窗口曲线

1. 标出初始 `cwnd`、`ssthresh` 和题目采用的 TCP 版本。
2. `cwnd < ssthresh` 时按题设慢启动增长。
3. 到阈值后按题设线性增长。
4. 超时：先更新阈值，再将窗口降到题设规定值。
5. 3 个重复 ACK：执行快重传/快恢复规则。
6. 每个 RTT 的“发送前还是发送后窗口”口径必须与题目一致。

11.7 交换机自学习

每帧都按固定顺序：

1. 先学习源 MAC 和入端口。
2. 再查目的 MAC。
3. 决定转发、过滤或泛洪。
4. 题目若给老化时间，删除过期表项。

11.8 Dijkstra

表格至少有三列：已确定集合 N'、各未确定节点当前最短距离、前驱。每轮选当前最小者加入 N'，再用它松弛邻居。若有同权路径，按题目规则或说明选择。

11.9 HTTP RTT 计算

先写清假设，再套公式：是否已有 DNS 缓存？是否已有 TCP 连接？是否 TLS？对象并行还是串行？传输时间是否忽略？

经典简化模型中，建立 TCP 约 1 RTT，发请求到收到很小对象首字节约 1 RTT；非持久串行获取 N 个对象可能对每个对象重复这两项。真实 HTTP/2/3、有缓存、连接复用和并发时不能机械使用旧公式。

11.10 考场检查清单

- 单位是否统一为 bit、s、Hz？
- dB 是否转成普通信噪比？
- 字节是否乘 8？
- IPv4 片偏移是否除以 8？
- 主机地址是否排除网络和广播地址？
- SYN/FIN 是否消耗序号？纯 ACK 是否携带数据？
- 流量控制与拥塞控制是否写反？
- 跨网段 ARP 是否查的是网关？
- 最长前缀匹配是否选最具体项？

第十二章 14 周学习路线与复习系统

12.1 推荐路线

周次	主题	必做产出
第 0 周	全局图、分层、封装、时延	手画五层封装图；第一次抓包
第 1 周	应用架构、Socket、HTTP	手写 HTTP 请求；分析浏览器 Network
第 2 周	DNS、缓存、CDN、邮件	画 DNS 查询时序；抓 DNS 包
第 3 周	端口、UDP、TCP 字节流	解释四元组；实现长度前缀思路
第 4 周	TCP 握手、挥手、可靠传输	抓握手与关闭；算 seq/ack
第 5 周	流量与拥塞控制	画 3 道 cwnd 曲线；辨认重传
第 6 周	IPv4、CIDR、子网	每天 3 道子网题；分析本机配置
第 7 周	分片、NAT、DHCP、ICMP、IPv6	做分片题；跑 ping/traceroute
第 8 周	路由算法与协议	手算 Dijkstra；对比 RIP/OSPF/BGP
第 9 周	以太网、交换机、ARP	模拟 MAC 表；抓 ARP
第 10 周	CRC、VLAN、物理层	手算 CRC；完成奈氏/香农题
第 11 周	Wi-Fi 与移动网络	分析“满格但慢”；画隐藏终端
第 12 周	TLS、证书和常见攻击	查看证书链；说明 HTTPS 边界
第 13 周	综合项目	迷你 HTTP 服务器或 TCP 聊天室
第 14 周	串联复习与模拟	15 分钟“网页之旅”毕业答辩

12.2 每周固定节奏

1. 周一至周三：学习新知识，每天只攻一个核心问题。
2. 周四：合上资料复述并画图，暴露记忆缺口。
3. 周五：做题，记录错误属于概念、计算、审题还是单位。

4. 周六：抓包或编程实验，把抽象字段变成真实证据。
5. 周日：写一页费曼笔记，用自己的话讲给“零基础同学”。

12.3 费曼笔记模板

```
主题：
它解决的问题：
没有它会发生的事故：
核心报文/字段：
正常时序：
丢包、超时或异常时怎样处理：
与上一层/下一层的关系：
一个真实案例：
我仍讲不清的地方：
```

12.4 最小项目：迷你 HTTP 服务器

项目应完成：

1. 监听本地端口并接受 TCP 连接。
2. 循环读取直到得到完整 HTTP 请求首部。
3. 解析请求行，支持 GET。
4. 返回 200 HTML 或 404。
5. 日志记录客户端 IP、端口、请求路径、状态码和耗时。
6. 设置请求大小和超时限制，避免无限等待。
7. 用 Wireshark 指出握手、请求、响应与关闭。

进阶：支持持久连接、线程池或 I/O 多路复用；再对照数据库服务器的 accept、缓冲、协议解析和并发模型。

12.5 毕业标准

- 能不看资料画出五层模型、封装与地址变化。

- 能解释 TCP 为什么可靠、为什么仍可能慢。
- 能在 3 分钟内完成常规 `/26`、`/27` 子网题。
- 能根据抓包判断握手、重传、零窗口、DNS 和 ARP。
- 能说明 HTTP/3 为什么运行在 UDP 之上仍可可靠。
- 能按层定位“能 ping 但网页打不开”。
- 能完整复述一次 HTTPS 网页访问，并指出哪些步骤可能命中缓存或被省略。

附录 A：常用命令与工具速查

Windows

```
ipconfig /all
Get-NetIPConfiguration
Get-NetTCPConnection
Get-NetUDPEndpoint
arp -a
route print
nslookup www.example.com
Resolve-DnsName www.example.com
ping www.example.com
tracert www.example.com
Test-NetConnection www.example.com -Port 443
```

Linux/macOS 常见替代

```
ip addr
ip route
ss -lntup
ip neigh
dig www.example.com
ping www.example.com
traceroute www.example.com
curl -v https://www.example.com/
```

命令输出与系统版本有关；不要只截图，要解释地址、接口、路由、端口和状态。

附录 B：章节过关问题参考答案要点

第一章

1. 分组交换共享链路；突发流量超过输出速率时队列溢出导致丢包。
2. 路由器去掉入链路帧并重新封装出链路帧；IP 数据报通常保留但 TTL 与校验和变化；TCP 报文段通常作为载荷不变，NAT/中间设备是例外。
3. 还可能是 RTT、丢包重传、拥塞、服务器处理、磁盘、无线干扰、DNS 或应用限速。
4. 发送时延取决于分组长度/链路速率；传播时延取决于距离/信号传播速度。

第二章

1. Cookie 在浏览器侧保存并随请求发送；Session 通常是服务器侧状态，二者可通过会话 ID 关联。
2. HTTP/2 仍受单条 TCP 连接丢包导致的跨流阻塞；QUIC 把可靠性放到独立流上。
3. 递归查询由被询问者继续得到最终结果；迭代查询返回下一步该问谁。
4. 方法表达语义，不提供机密性；GET/POST 都需要 HTTPS 和正确鉴权。
5. 客户端主动连接服务器在被动响应中给出的临时端口。

第三章

1. TCP 只承诺有序字节流；应用用长度、分隔符、固定长度或自描述解析定义消息。
2. 建立要让双方确认收发能力和初始序号；关闭的两个方向可独立结束。
3. 若 `seq=1000` 指向首字节并收到 500 B，累计确认号为 1500。
4. `rwnd` 防接收方缓冲溢出，`cwnd` 防网络拥塞。
5. 重复 ACK 说明部分数据仍在流动，超时更像严重拥塞，因此降速幅度不同。

第四章

1. `/20` 掩码 `255.255.240.0`，网络 `172.16.32.0`，广播 `172.16.47.255`。
2. 链路层只能把帧交给本地链路下一跳；跨网目标要先交给网关。
3. 任一分片丢失都可能让整个原始数据报无法重组，上层还会重传完整数据。
4. 先选所有匹配项中前缀最长者；无更具体项时 `/0` 默认路由兜底。
5. NAT 只有已有映射才能把返回流量送到内网，外部不知道应映射到哪台主机和端口。

第五章

1. 路由协议形成路由信息，设备据此安装用于高速查找的转发表。
2. 节点只从邻居逐步获知故障，错误距离可能相互引用并缓慢增大。
3. OSPF 管 AS 内部并偏向最短路径；BGP 管 AS 之间并强调策略和可扩展性。
4. 本地优先级、商业关系、出口策略等属性可优先于 AS 路径长度。

第六章

1. 学源 MAC，因为入端口证明“这个源可以从这里到达”。
2. 未知单播的地址仍是单播，只因表中未知而泛洪；广播帧本身就是发给广播地址。
3. B 不在本链路，第一跳必须交给能继续路由的默认网关。
4. 每个端口形成独立全双工链路，不再与其他发送者竞争同一冲突域。
5. VLAN 划分二层广播域；不同 VLAN 通过路由器或三层交换机转发。

第七章

1. 一个符号承载多比特时，比特率可高于波特率。
2. 一个限制无噪声符号变化，一个限制有噪声信息容量，系统必须同时满足。
3. 有线可在发送时检测碰撞；无线自干扰和隐藏终端使检测困难，所以尽量避免并用 ACK 推断成功。

4. 缓解隐藏终端；控制帧开销可能大于收益，所以按阈值或环境选择。

第八章

1. 需要安全建立共享密钥并验证身份；大量数据随后用更快的对称加密。
2. HMAC 要求验证者共享秘密；签名允许持公钥的任何人验证而只有私钥持有者能签。
3. 通常不能隐藏目标 IP，也不能阻止合法服务器滥用数据；HTTPS 主要保护传输链路。
4. 攻击者若没有可信 CA 对目标域名签发的有效证书，客户端验证会失败并报警。

附录 C：核心术语速查

术语	一句话解释
RTT	一次往返所需时间
MTU	一条链路可承载的最大网络层分组大小上限（具体定义依链路）
MSS	TCP 报文段希望承载的最大数据长度，不含 TCP/IP 首部
TTL / Hop Limit	限制分组可经过的路由跳数
四元组	源 IP、源端口、目的 IP、目的端口
吞吐量	应用实际获得的有效传输速率
抖动	时延随时间的波动
ARP	在本地 IPv4 链路上把下一跳 IP 解析成 MAC
NAT/PAT	改写地址/端口并维护映射，让多个内网主机共享公网地址
CIDR	用前缀长度描述网络并支持聚合的无类编址
cwnd	发送端根据网络拥塞状态维护的窗口
rwnd	接收端通告的可接收窗口
CDN	把内容分布到靠近用户的边缘节点
TLS	为上层通信提供加密、完整性和身份认证的安全协议
QUIC	基于 UDP、集成 TLS、支持多流的现代传输协议
VLAN	在二层网络中划分逻辑广播域

最后一句： 计算机网络不是背端口号的学科，而是研究“信息怎样在不可靠、共享、分层的系统中仍然按时、按序、安全地到达”的学科。每当你能从一次真实抓包解释一个字段，就把抽象协议变成了自己的知识。